V-Platform — Complete Architecture Report

> ***Project***: *V-Platform (Multiplayer Game Backend + Admin Dashboard)* ***Date***: *March 24, 2026* ***Version***: *1.0*

Table of Contents

1. Executive Summary
2. System Overview
3. V-Server — Backend Architecture
4. V-Dashboard — Frontend Architecture
5. Authentication & Authorization
6. Game Flow — End to End
7. Data Models
8. API Reference
9. Infrastructure & Cloud Services
10. Logging & Monitoring
11. Scheduled Jobs
12. Tech Stack Summary

---

1. Executive Summary

V-Platform is a **multiplayer game platform** consisting of two tightly coupled projects:

Component	Description	Tech Stack
V-Server	Game backend server handling matchmaking, lobbies, game server orchestration, authentication, and admin APIs	Node.js + Fastify + MongoDB + Redis + Elasticsearch
V-Dashboard	Admin/developer dashboard for managing game entities, monitoring live sessions, deploying builds, and toggling feature flags	Next.js 14 + React 18 + TypeScript + Tailwind CSS

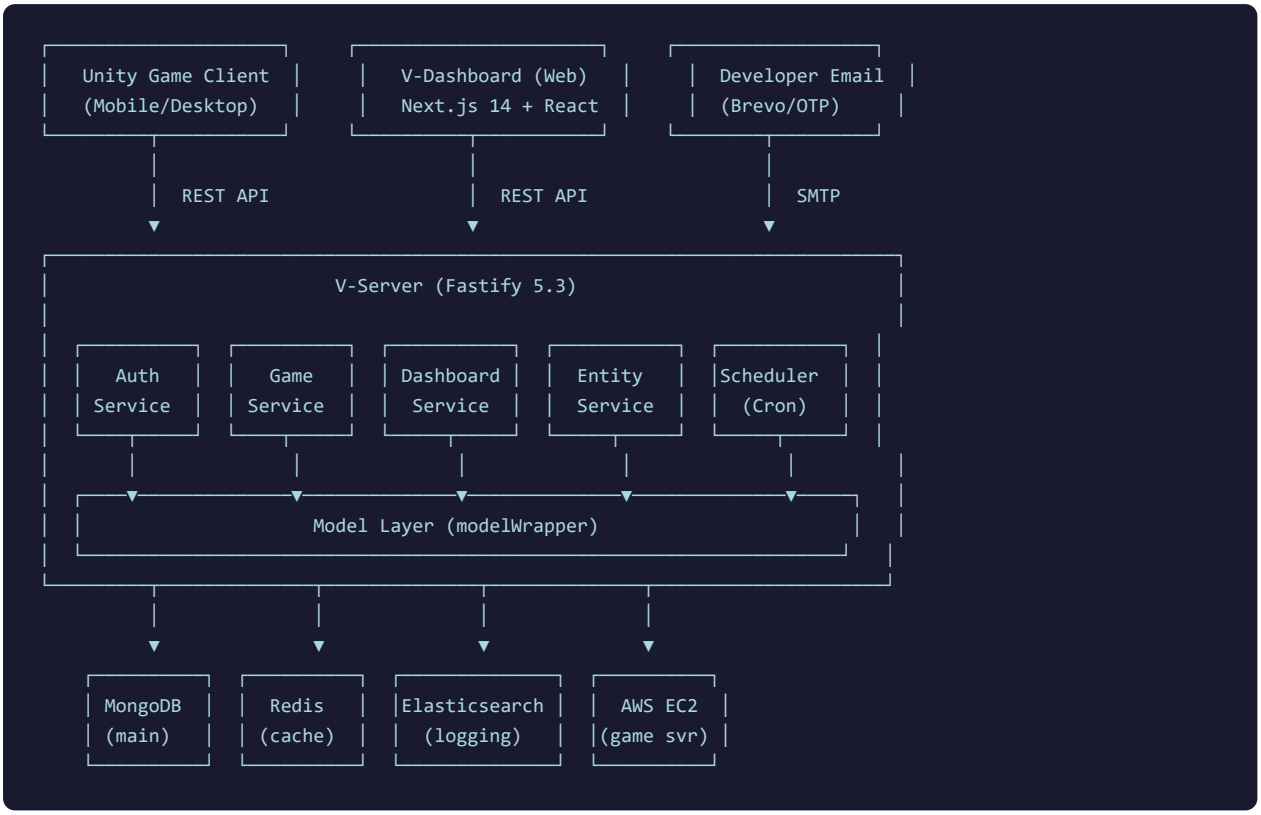
The platform supports real-time multiplayer games with automated AWS EC2 game server provisioning, OTP-based authentication, and comprehensive admin tooling.

Scale: ~74 backend files across 19 directories | ~149 frontend files with ~18,900 lines of code

---

2. System Overview

High-Level Architecture



Communication Patterns

From	To	Protocol	Purpose
Unity Client → V-Server	REST	Matchmaking, lobby, character selection, logs	
V-Dashboard → V-Server	REST	Entity CRUD, session monitoring, admin ops	
EC2 Game Server → V-Server	REST (server-to-server)	Kill events, game over, game data, build status	
V-Server → AWS EC2	AWS SDK	Spawn/terminate game server instances	
V-Server → Brevo	SMTP API	Send OTP emails to developers	
V-Server → Elasticsearch	HTTP	Centralized log ingestion	

3. V-Server — Backend Architecture

3.1 Project Structure

```
V-Server/
├── index.js                    # Entry point — Fastify server setup
├── package.json                # Dependencies & scripts
├── .env                        # Environment variables
│
├── constants/                  # Configuration & constants
│   ├── config.js               # Environment-based config
│   └── constants.js            # App-wide constants
│
├── db/                         # Database connections
│   ├── mongo.js                # MongoDB (Mongoose) connection
│   ├── redis.js                # Redis client with reconnection
│   └── elastic.js              # Elasticsearch client
│
├── model/                      # Mongoose schemas (17 models)
│   ├── modelWrapper.js         # Logging/metrics wrapper for all models
│   ├── User.js                 # Game user
│   ├── Developer.js            # Admin/developer user
│   ├── UserOtp.js              # User OTP (10-min TTL)
│   ├── DeveloperOtp.js         # Developer OTP (10-min TTL)
│   ├── UserProfile.js          # Player profile (character, name)
│   ├── Character.js            # Game character (stats, abilities)
│   ├── Building.js             # Buildable structure
│   ├── Inventory.js            # Inventory item
│   ├── Lobby.js                # Custom game lobby
│   ├── Game.js                 # Matchmade game
│   ├── CustomGame.js           # Custom game (from lobby)
│   ├── Build.js                # Server build
│   ├── ClientBuild.js          # Client build
│   ├── FeatureFlag.js          # Feature toggles
│   ├── GlobalConfig.js         # Key-value config
│   ├── DeviceData.js           # Client device info
│   └── Query.js                # Raw query model
│
├── route/                      # Fastify route handlers
│   ├── auth/                   # Authentication routes
│   │   ├── user.js             # User login/OTP
│   │   ├── developer.js        # Developer login/OTP
│   │   └── refreshToken.js     # Token refresh
│   ├── game/                   # Game routes
│   │   ├── matchmaking.js      # Find match, get status, cancel
│   │   ├── lobby.js            # Lobby CRUD, start match
│   │   ├── chooseCharacter.js  # Character selection
│   │   ├── killPlayer.js       # Player death (server-to-server)
│   │   ├── gameOver.js         # Game end (server-to-server)
│   │   ├── gameData.js         # Game state data
│   │   ├── info.js             # Game info (characters, buildings)
│   │   ├── unityLogs.js        # Client log ingestion
│   │   └── userProfile.js      # Profile management
│   ├── dashboard/              # Admin dashboard routes
│   │   ├── build.js            # Server build management
│   │   ├── clientBuild.js      # Client build management
│   │   ├── developer.js        # Developer account CRUD
│   │   ├── featureFlag.js      # Feature flag CRUD
│   │   ├── whitelistedUser.js  # Whitelist management
│   │   ├── excalidrawBoard.js  # Whiteboard boards
│   │   ├── queryService.js     # Raw MongoDB queries
│   │   └── gameSessions.js     # Session monitoring
│   ├── entity/                 # Entity CRUD routes
│   │   ├── character.js        # Character CRUD
```

```
│   │   ├── building.js         # Building CRUD
│   │   ├── inventory.js        # Inventory CRUD
│   │   └── globalConfig.js     # Config CRUD
│   └── docs/                   # API documentation
│
├── service/                    # Business logic layer
│   ├── auth/                   # Auth services
│   ├── game/                   # Game logic services
│   ├── dashboard/              # Dashboard services
│   ├── ec2/                    # EC2 manager & auto-shutdown
│   ├── log/                    # Logging service (Elasticsearch)
│   └── brevoService.js         # Email (OTP) service
│
├── scheduler/                  # Cron jobs
│   ├── index.js                # Job registry
│   ├── lobby.js                # Lobby cleanup (every 30 min)
│   ├── gameServer.js           # Stale game cleanup (every 1 min)
│   └── elasticsearch.js        # ES index maintenance
│
├── util/                       # Utilities
│   └── middleware.js            # Guards, wrapper, parseAppConfig
│
└── lib/                        # Backend data utilities
```

3.2 Server Startup Flow

```
index.js
  │
  ├── 1. Create Fastify instance (body limit: 1MB)
  ├── 2. Connect MongoDB (database: "main")
  ├── 3. Connect Redis (exponential backoff reconnection, max 3s)
  ├── 4. Register CORS (allowed origins for dashboard & dev)
  ├── 5. Register fastify-multipart (10MB file limit)
  ├── 6. Add global preHandler: parseAppConfig (JWT decode on every request)
  ├── 7. Register all route handlers under /api prefix
  ├── 8. Listen on PORT (default 5000), host 0.0.0.0
  └── 9. Global error handlers: unhandledRejection + uncaughtException → log + exit
```

3.3 Layered Architecture



4. V-Dashboard — Frontend Architecture

4.1 Project Structure

```
v-dashboard/
├── app/                          # Next.js App Router
│   ├── layout.tsx                # Root layout (ThemeProvider, Navbar, Toaster)
│   ├── page.tsx                  # Root — redirect to /dashboard or /sign-in
│   ├── globals.css               # Global styles
│   │
│   ├── (auth)/(routes)/          # Auth route group
│   │   ├── sign-in/page.tsx      # OTP login page
│   │   └── register/page.tsx     # Registration (commented out)
│   │
│   ├── dashboard/page.tsx        # Main hub — sessions, stats
│   ├── characters/               # Character entity CRUD
│   │   ├── page.tsx              # List all characters
│   │   ├── new/page.tsx          # Create character form
│   │   └── [characterId]/        # Edit & details
│   ├── buildings/                # Building entity CRUD
│   ├── inventory/                # Inventory entity CRUD
│   ├── global-config/            # Config entity CRUD
│   ├── feature-flags/page.tsx    # Feature flag management
│   ├── developers/page.tsx       # Developer accounts (admin)
│   ├── whitelist/page.tsx        # Whitelist management (admin)
│   ├── queries/page.tsx          # MongoDB/Redis query tool (admin)
│   ├── builds/page.tsx           # Server build management (prod)
│   ├── clientBuilds/page.tsx     # Client build management (prod)
│   ├── excalidraw/               # Collaborative whiteboard
│   │   ├── page.tsx              # Board list
│   │   ├── new/page.tsx          # Create board
│   │   └── [boardId]/page.tsx    # Board editor
│   ├── system-design/page.tsx    # Architecture visualization
│   └── d/                        # Public downloads
│
├── components/
│   ├── ui/                       # ~30 shadcn/ui components
│   ├── navbar.tsx                # Main navigation
│   ├── theme-provider.tsx        # Dark/light mode
│   ├── theme-toggle.tsx          # Theme switcher
│   ├── server-toggle.tsx         # Dev/prod API toggle
│   ├── ability-modal.tsx         # Ability editor
│   └── prod-only-guard.tsx       # Production page guard
│
├── hooks/                        # Custom hooks
│   ├── use-flow-data.ts          # System design diagram data
│   ├── use-origin.tsx            # Environment detection
│   └── use-store-modal.tsx       # Modal state
│
├── lib/
│   ├── auth.ts                   # JWT auth service (login, refresh, fetch wrapper)
│   ├── api-config.ts             # API base URL (dev/prod)
│   ├── Types.ts                  # TypeScript interfaces
│   ├── backend-data.ts           # Backend endpoint documentation
│   ├── network-data.ts           # Network diagram data
│   ├── feature-flag.ts           # Feature flag checker
│   └── utils.ts                  # cn() helper
│
├── middleware.ts                  # Next.js middleware (auth validation)
├── tailwind.config.ts            # Tailwind + custom theme
├── next.config.mjs               # Next.js config (Cloudinary images)
└── package.json
```

4.2 Key UI Patterns

Pattern	Implementation
Component Library	shadcn/ui (~30 components) built on Radix UI + Tailwind
Forms	React Hook Form + Zod schema validation
Data Tables	Custom DataTable with search, sort, pagination, row actions
Notifications	Sonner toast library with rich colors
Icons	lucide-react (427 icons)
Theming	next-themes with dark/light mode, CSS variable color system
State	Local useState + localStorage (no global store)
API Calls	authService.authenticatedFetch() with auto token refresh

4.3 Page Capabilities

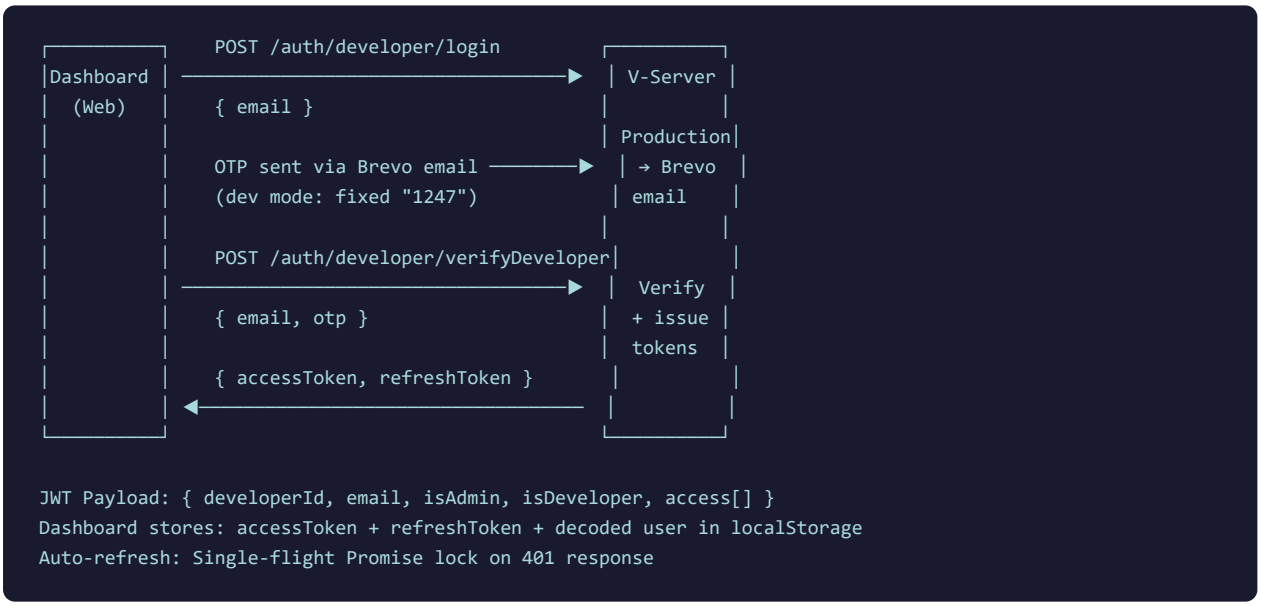
Page	Key Features
Dashboard	Live game sessions, active lobbies, running/completed games, 30s auto-refresh
Characters	Full CRUD, complex ability trees (4-key binding), stats editing
Buildings	CRUD with health, attack, storage, fireRate, bulletSpeed fields
Inventory	CRUD with cost, duration, storage, abilities
Feature Flags	Toggle flags (STRING/BOOLEAN/INTEGER), create/delete
Builds	Server/client build lifecycle management (prod-only)
Queries	Raw MongoDB/Redis query execution with JSON result viewer
System Design	3-tab architecture visualization (Unity/Backend/VCM) using React Flow
Excalidraw	Collaborative whiteboarding with board CRUD

5. Authentication & Authorization

5.1 User Authentication (Game Players)



5.2 Developer Authentication (Dashboard)



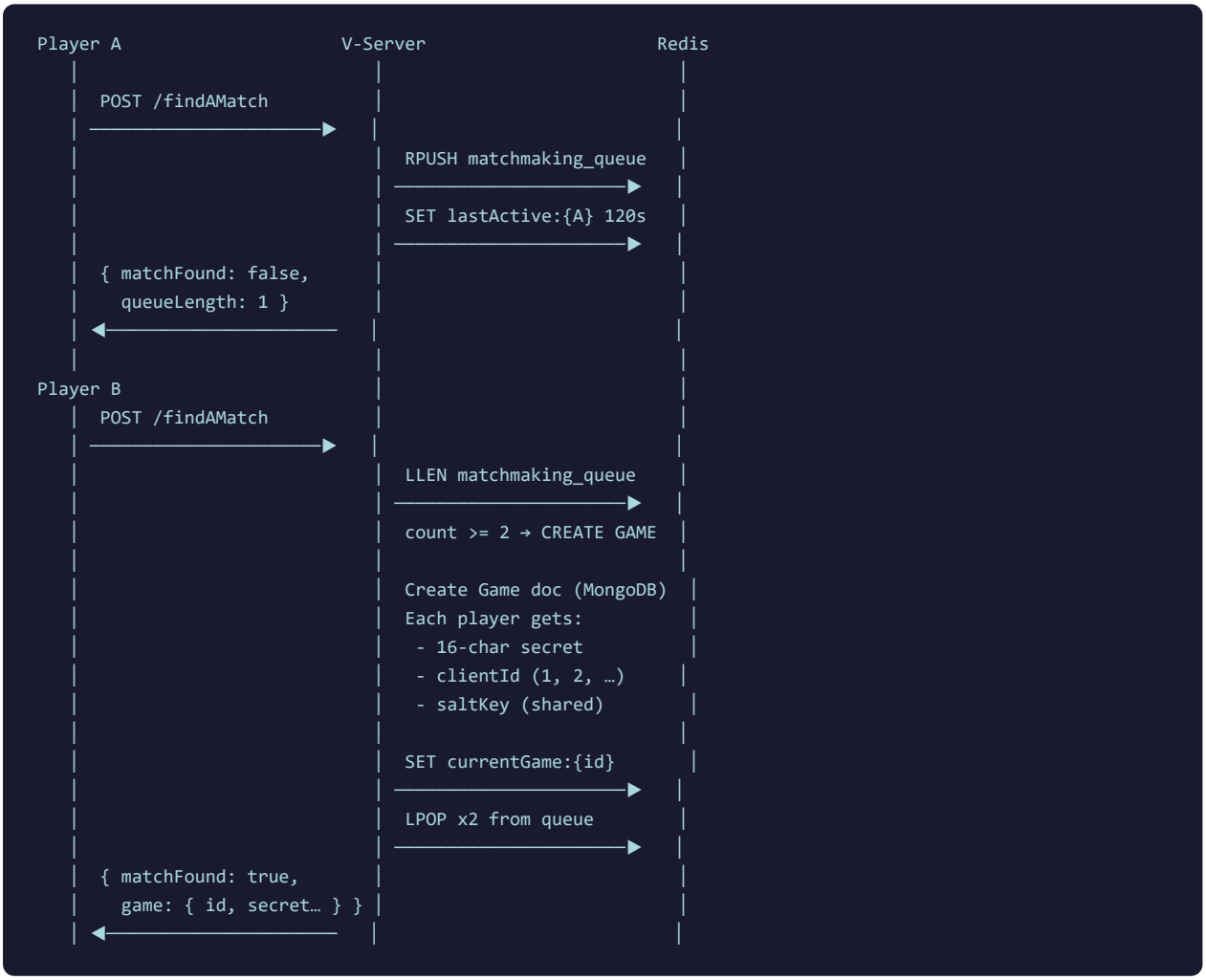
5.3 Authorization Guards

Guard	Check	Used For
`requireUser`	isAdmin || isDeveloper || isUser	General authenticated endpoints
`requireDeveloper`	isAdmin || isDeveloper	Entity management, feature flags
`requireAdmin`	isAdmin	Developer CRUD, whitelist, queries
`requireServer`	x-server-secret header = SERVER_SECRET	EC2 game server → backend calls
`requireVcmUser`	isAdmin || isVcmUser	VCM-specific features
`checkAccess(perm)`	developer.access includes perm	Granular feature access

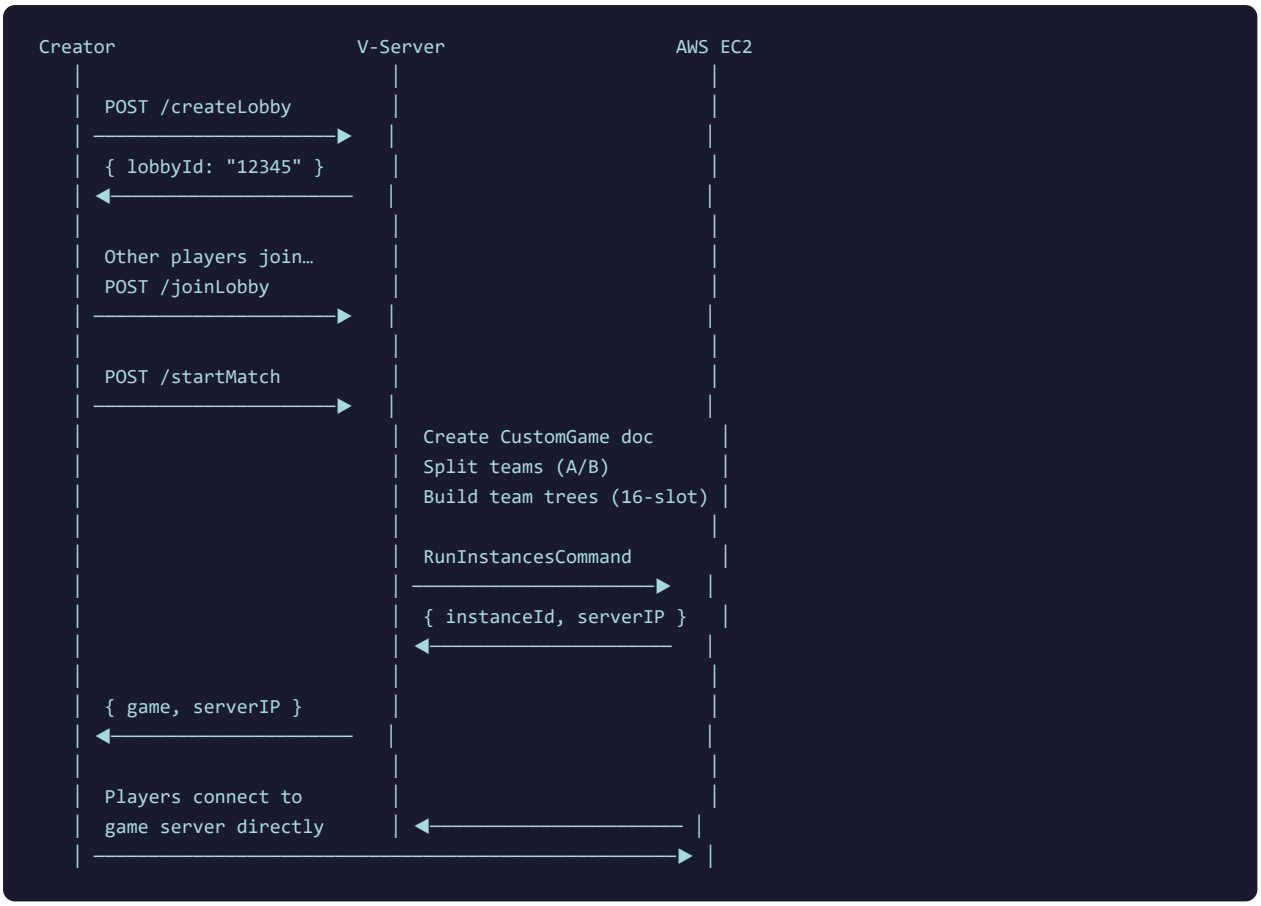
---

6. Game Flow — End to End

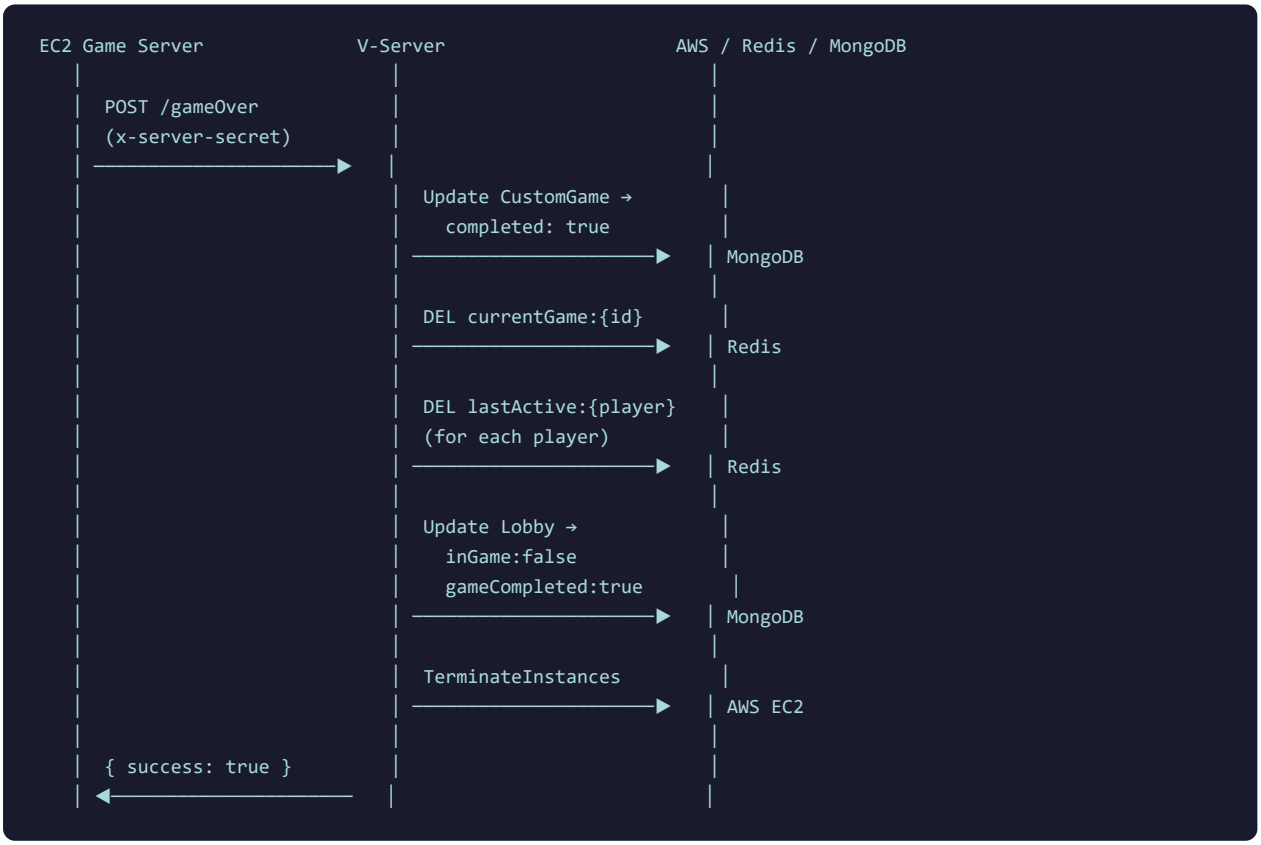
6.1 Matchmaking Flow



6.2 Lobby Flow



6.3 Game Over Flow



6.4 Team Tree Algorithm

The team distribution uses a flat array to position players in a 16-slot hierarchy:

```
Flat Tree: [1, 2, 3, 8, 4, 11, 14, 6, 5, 12, 15, 7, 9, 13, 16, 10]

Player at index 0 → localTeamId=1 → position=0  → filledTree[0] = clientId
Player at index 1 → localTeamId=2 → position=1  → filledTree[1] = clientId
Player at index 2 → localTeamId=3 → position=2  → filledTree[2] = clientId
…and so on

Result: 16-element array with player clientIds in strategic positions
```

---

7. Data Models

7.1 Core Game Models



7.2 Game Session Models



7.3 Admin Models



8. API Reference

8.1 Authentication APIs

Method	Endpoint	Auth	Description
POST	`/api/auth/user/login`	None	Send OTP to user email
POST	`/api/auth/user/verifyOtp`	None	Verify OTP, return tokens
POST	`/api/auth/developer/login`	None	Send OTP to developer email
POST	`/api/auth/developer/verifyDeveloper`	None	Verify developer OTP
POST	`/api/auth/refresh/newToken`	Refresh Token	Get new access token

8.2 Game APIs

Method	Endpoint	Auth	Description
POST	`/api/game/matchmaking/findAMatch`	User	Join matchmaking queue
POST	`/api/game/matchmaking/getStatus`	User	Check match status
POST	`/api/game/matchmaking/cancelSearch`	User	Leave queue
POST	`/api/game/lobby/createLobby`	User	Create custom lobby
POST	`/api/game/lobby/joinLobby`	User	Join a lobby
POST	`/api/game/lobby/leaveLobby`	User	Leave a lobby
POST	`/api/game/lobby/startMatch`	User	Start game from lobby
POST	`/api/game/lobby/getLobbyStatus`	User	Get lobby info
POST	`/api/game/lobby/getActiveGame`	User	Get active game
POST	`/api/game/chooseCharacter`	User	Select character
POST	`/api/game/killPlayer`	Server	Player death event
POST	`/api/game/gameOver`	Server	End game session
POST	`/api/game/gameData`	Server	Game state data
POST	`/api/game/unityLogs/logs`	User	Client log ingestion
POST	`/api/game/unityLogs/errors`	User	Client error ingestion

8.3 Game Info APIs

Method	Endpoint	Auth	Description
GET	`/api/game/info/characters`	None	All active characters
GET	`/api/game/info/buildings`	None	All active buildings
GET	`/api/game/info/inventory`	None	All active inventory
GET	`/api/game/info/global-config`	None	Global config entries
POST	`/api/game/info/serverData`	Server	Full game data package

8.4 Entity CRUD APIs

Method	Endpoint	Auth	Description
GET	`/api/entity/character`	Developer	List/get characters
POST	`/api/entity/character`	Developer	Create character
PUT	`/api/entity/character`	Developer	Update character
DELETE	`/api/entity/character`	Developer	Delete character
GET	`/api/entity/building`	Developer	List/get buildings
POST	`/api/entity/building`	Developer	Create building
PUT	`/api/entity/building`	Developer	Update building
DELETE	`/api/entity/building`	Developer	Delete building
GET	`/api/entity/inventory`	Developer	List/get inventory
POST	`/api/entity/inventory`	Developer	Create inventory item
PUT	`/api/entity/inventory`	Developer	Update inventory item
DELETE	`/api/entity/inventory`	Developer	Delete inventory item
GET	`/api/entity/global-config`	Developer	List/get config
POST	`/api/entity/global-config`	Developer	Create config
PUT	`/api/entity/global-config`	Developer	Update config
DELETE	`/api/entity/global-config`	Developer	Delete config

8.5 Dashboard APIs

Method	Endpoint	Auth	Description
POST	`/api/dashboard/build/createBuild`	Admin	Start server build
GET	`/api/dashboard/build/getBuilds`	Developer	List builds
POST	`/api/dashboard/build/setActive`	Admin	Activate a build
GET	`/api/dashboard/build/getCurrentStatus`	Developer	Build status from Redis
POST	`/api/dashboard/build/getLogs`	Developer	Build logs from ES
POST	`/api/dashboard/featureFlag/*`	Developer+	Feature flag CRUD
POST	`/api/dashboard/query/execute`	Admin	Raw MongoDB query
POST	`/api/dashboard/gameSessions/getAll`	Developer	Live session data
GET	`/api/dashboard/developer/list`	Admin	List developers
POST	`/api/dashboard/whitelist/*`	Admin	Whitelist management
POST	`/api/dashboard/excalidraw/*`	Developer	Whiteboard CRUD

---

9. Infrastructure & Cloud Services

9.1 Database Infrastructure

Database	Purpose	Key Data
MongoDB	Primary data store	Users, games, characters, buildings, inventory, builds, feature flags
Redis	Cache & real-time state	Matchmaking queue, game caches, heartbeats, build status, feature flags, lobby data
Elasticsearch	Centralized logging	Application logs, error logs, build logs, client logs

9.2 Redis Key Patterns

Key Pattern	Type	TTL	Purpose
`matchmaking_queue`	List	—	Player matchmaking queue
`currentGame:{gameId}`	String (JSON)	—	Active game cache
`lastActive:{userId}`	String	120s	Player heartbeat
`lobby:{lobbyId}`	Hash	2h	Lobby data
`userLobby`	Hash	—	User → lobby mapping
`server_build_status`	Hash	—	Current build state
`featureFlags`	Hash	—	Feature flag cache

9.3 AWS Services

Service	Usage
EC2	Spawn game server instances per match, Unity build server management
S3	Store and retrieve game build artifacts
CloudFront	CDN for asset delivery

9.4 External Services

Service	Usage
Brevo (Sendinblue)	Send OTP emails to developers
Firebase Admin	Cloud notifications (prepared, not heavily used)
Cloudinary	Image hosting for dashboard (Next.js image optimization)

---

10. Logging & Monitoring

10.1 Log Architecture



10.2 Log Categories

Category	Source	Description
`LOG`	Server	Application-level logs
`ERROR`	Server	Errors with stack traces
`EVENT`	Server	Business events
`FE_LOG`	Unity Client	Client-side logs sent via `/unityLogs`

10.3 Log Fields

```
{
  "source": "server",
  "timestamp": "auto",
  "txnId": "unique-request-id",
  "key1": "feature/endpoint",
  "key2": "action/phase",
  "text1": "human-readable description",
  "text2": "additional context",
  "data": "request/response payload",
  "error": "error message",
  "stack": "stack trace",
  "metric": "execution time (ms)"
}
```

---

11. Scheduled Jobs

Job	Schedule	Purpose
Lobby Cleanup	Every 30 minutes	Remove stale lobbies where creator is inactive 30+ min
Game Server Cleanup	Every 1 minute	Terminate games older than 20 min, cleanup EC2/Redis/MongoDB
Elasticsearch Maintenance	Configurable	Manage old log indices

Game Server Cleanup Detail

```
Every 1 minute:
  1. Query CustomGame: { completed: false, createdAt < 20 min ago }
  2. For each stale game:
     a. Terminate EC2 instances (production)
     b. Delete Redis cache: currentGame:{id}
     c. Delete lastActive keys for all players
     d. Set CustomGame.completed = true
  3. Handle already-terminated instances gracefully
```

---

12. Tech Stack Summary

V-Server (Backend)

Category	Technology	Version
Runtime	Node.js (ES Modules)	—
Framework	Fastify	5.3.3
Database	MongoDB (Mongoose)	8.15.1
Cache	Redis	5.8.2
Search/Logs	Elasticsearch	8.19.1
Auth	JWT (jsonwebtoken)	—
Cloud	AWS SDK (EC2, S3)	v3
Email	Brevo (Sendinblue)	3.0.1
Push	Firebase Admin	13.4.0
Scheduler	node-cron	4.2.1
File Upload	fastify-multipart	—

V-Dashboard (Frontend)

Category	Technology	Version
Framework	Next.js	14.2.32
UI Library	React	18
Language	TypeScript	5
Styling	Tailwind CSS	3.4.1
Components	shadcn/ui (Radix UI)	~30 components
Forms	React Hook Form + Zod	—
Icons	lucide-react	—
Diagrams	@xyflow/react + dagre	12.9.2
Whiteboard	@excalidraw/excalidraw	0.18.0
Charts	Recharts	3.5.0
Toasts	Sonner	2.0.7
Theming	next-themes	0.4.6

13. About the Team

The Builders

This entire platform — backend, frontend, infrastructure, and game logic — was built by a **2-person team**.

	Developer	Lead / Senior
Age	19	23
Role	Primary executor — wrote and shipped all the code	Architecture, mentorship, guidance, code review
Background	5 years of programming (started at 14), martial arts practitioner (5 years)	B.Tech from DTU (Delhi Technological University), ex-Urbanworks Senior Engineer, 4 years industry experience
Split	~50% execution, hands-on building	~50% teaching, design decisions, architecture

What This Represents

For a team of two — one learning, one guiding — this project demonstrates:

- **Production-grade architecture**: Clean layered separation (route → service → model), not a tutorial project
- **Distributed systems thinking**: Dual-write feature flags (Redis + MongoDB), Redis-based matchmaking queues, AsyncLocalStorage request tracing
- **Cloud infrastructure**: Real AWS EC2 game server orchestration — spawning and terminating instances per match
- **Full-stack delivery**: Backend API server + Admin dashboard + Game server integration + Cron-based cleanup — end to end
- **Multiple database mastery**: MongoDB for persistence, Redis for real-time state, Elasticsearch for centralized logging — each used for its strength

Engineering Assessment

Aspect	Rating	Notes
Architecture & code separation	Senior-level	Clean layers, proper abstraction
Auth system	Production-ready	JWT + OTP + role-based guards
Game flow design	Well-engineered	Matchmaking, lobbies, team trees, lifecycle management
Cloud infrastructure	Beyond typical junior scope	EC2 orchestration, S3, CloudFront
Admin tooling	Complete	Entity CRUD, feature flags, live monitoring, query tool
Real-time state management	Solid	Redis for queues, heartbeats, caches
Logging & observability	Professional	Elasticsearch + txnId tracing + metrics

> *"Most developers at 19 are building todo apps. This team built a multiplayer game platform with AWS game server orchestration, real-time matchmaking, and a full admin dashboard."*

> ***End of Report Built by**: Aswin (Developer) & Senior Lead **Generated**: March 24, 2026*